Scheduling snapshot restores along a workflow

Theory chapter, draft

MSc thesis, Amirkabir University of Technology

October 8, 2026

Chapter 4

Scheduling snapshot restores along a workflow

A snapshot system restores a function when its request arrives. In a workflow, a stage's request arrives only when its predecessors finish, so a cold workflow of d stages pays d restores one after another. This chapter develops the theory of the alternative: the orchestrator, which knows the workflow graph and each function's measured timings, restores every stage *ahead of need*, each timed to be ready when its input arrives. The workflow's own upstream execution then hides the downstream restores.

The chapter answers four questions and then shows how they fit together:

1. **When to restore each stage** (Sections 4.2 and 4.3). Restoring each stage just in time reaches the lowest latency any policy can reach, and holds no more memory·time than restoring on demand (Theorems 4.4 and 4.6). Uncertain run times and if/else branches are governed by one price ratio (Theorems 4.9 and 4.10).

2. **Which stages get a snapshot, and how deep** (Section 4.5). Under look-ahead, restore time no longer adds up along a path, so the choice needs a new objective. Two numbers per sub-workflow suffice, and a dynamic program over series-parallel workflows finds the exact optimum (Theorem 4.13).

3. **Memory** (Section 4.6). Look-ahead raises the memory peak. With a budget of k sandboxes on a chain each extra sandbox divides the restore cost, and a memory guard never exceeds the budget and never deadlocks (Theorem 4.17).

4. **How much CPU each start-up gets** (Section 4.9). A start-up is CPU work, and small CPU shares starve it. The server's spare CPU is planned per start-up and over time: every start-up has a deadline from the workflow graph, and the exact plan is a maximum flow (Theorems 4.25 and 4.26). The fixed-CPU look-ahead above is the case with no spare CPU.

Section 4.7 shows that the whole problem is an instance of a classical operations-research problem, resource-constrained project scheduling with time lags. The results above are its tractable special cases; the general memory-limited case is NP-hard and is solved by a textbook branch and bound for workflows of realistic size. The section ends with the algorithm the platform runs. Section 4.10 describes how every result was checked by computation, including checks designed to fail.

4.1 Model

Definition 4.1 (Workflow)**.** A workflow is a directed acyclic graph $G = (V, E)$ with a single entry stage e from which every stage is reachable. Its stages are numbered in topological order.

An edge (u, v) means that stage v needs the output of u; a stage with several predecessors waits for all of them (an AND-join). If/else branches are treated in Section 4.3. Each stage v has

- a *restore time* $r_v > 0$: the time from starting the restore of its snapshot to a sandbox ready to accept a request;
- a *warm work* $w_v \geq 0$: the time the stage takes once it starts on a restored sandbox, that is the residual warm-up the snapshot did not capture plus the steady-state execution;
- a *memory* $m_v > 0$ held by its sandbox.

Passing an output along an edge takes a constant $\delta \geq 0$.

A *cold workflow* arrives at time 0 and finds no live sandbox for any of its stages. A *policy* chooses, for each stage v, a *trigger* τ_v: the time at which the restore of v starts.

Definition 4.2 (Schedule)**.** For triggers τ, the input time I_v, start S_v and finish F_v of each stage are

$$I_e = 0, \qquad I_v = \max_{u \in \mathrm{pred}(v)} F_u + \delta \quad (v \neq e),$$
$$S_v = \max(\tau_v + r_v,\ I_v), \qquad F_v = S_v + w_v.$$

The *latency* is $L = \max_{v \text{ a sink}} F_v$ and the *memory·time* is $M = \sum_v m_v(F_v - \tau_v)$: stage v holds m_v from its trigger until it finishes.

Three policies recur throughout.

- **On demand**: $\tau_v = I_v$. The restore starts when the input arrives. This is what single-function snapshot systems do today.
- **Eager**: $\tau_v = 0$. Every restore starts when the workflow arrives.
- **Just in time (JIT)**: $\tau_v = S_v^{\text{eager}} - r_v$, where S^{eager} is the start time under the eager policy. Each restore finishes exactly when the stage can start.

For a path P let $W(P) = \sum_{v \in P} w_v + \delta\,(|P| - 1)$ be its warm length, let $\ell(v)$ be the largest $W(P)$ over paths from v to a sink, and let $\ell'(v)$ be the same with node weights $r_v + w_v$ in place of w_v.

Table 4.1: Assumptions of the model.

assumption	statement	status
A1 no clairvoyance	$\tau_v \geq 0$: nothing starts before the workflow arrives	conservative: predicting arrivals could only help (Theorem 4.4's control shows the bound needs A1)
A2 independent restores	a restore triggered at τ is ready at $\tau + r_v$	relaxed in Theorem 4.7 (contention) and Theorem 4.8 (random r_v); to be measured
A3 fixed work	a stage takes w_v once started	re-warming would lower w_v; to be measured
A4 memory	stage v holds m_v from τ_v to F_v	lazy restore would hold less, so A4 is conservative for memory

Example 4.3 (Running example)**.** A chain of three Java stages with $r = 650\,\text{ms}$, $w = 75\,\text{ms}$ (63 ms of residual warm-up plus 12 ms of steady execution), $\delta = 2\,\text{ms}$ and $m = 512\,\text{MB}$, the values used in the simulations of this thesis. On demand, each stage adds $r + w + \delta$: $L = 3 \cdot 725 + 2 \cdot 2 = 2179\,\text{ms}$. Just in time, the triggers are 0, 77 and 154 ms, each stage adds only $w + \delta$ after the first restore, and $L = 650 + 3 \cdot 75 + 2 \cdot 2 = 879\,\text{ms}$ (Figure 4.1). Both hold 1.11 GB·s of memory·time; eager holds 1.23 GB·s. The price is the peak: 1.5 GB at once instead of 0.5 GB.

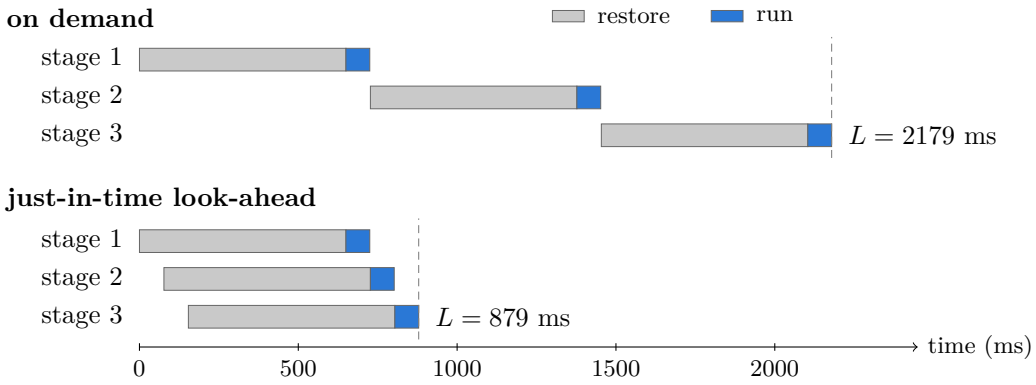


Figure 4.1: The running example (Theorem 4.3). On demand, each restore starts when the stage's input arrives, and the three restores add up. Just in time, each restore starts r before the stage can start; the upstream work hides all restores but the first.

4.2 Look-ahead restore is optimal

Theorem 4.4 (Look-ahead restore is latency-optimal)**.** *Under A1–A3:*

(a) every policy has latency $L \geq L^* := \max_{v \in V}(r_v + \ell(v))$*;*

(b) the eager policy attains L^**;*

(c) the on-demand policy has latency $L_{\text{od}} = \ell'(e)$*, the longest path with node weights* $r + w$*;*

(d) $L^* \leq L_{\text{od}}$*.*

Proof. (a) Fix a stage v and a path $P = (v_0 = v, v_1, \ldots, v_k)$ from v to a sink. By A1 and A2, $S_{v_0} \geq \tau_{v_0} + r_{v_0} \geq r_{v_0}$. For $i \geq 1$, $S_{v_i} \geq I_{v_i} \geq F_{v_{i-1}} + \delta$. Adding up along the path, $F_{v_k} \geq r_{v_0} + W(P)$, and $L \geq F_{v_k}$. Taking the maximum over P and v gives the bound.

(b) Under the eager policy $S_v = \max(r_v, I_v)$. By induction in topological order, $F_v = \max_{u,P}(r_u + W(P))$ over the ancestors u of v (including v) and the paths P from u to v: the term $r_v + w_v$ covers $u = v$, and $\max_{u \in \text{pred}(v)} F_u + \delta + w_v$ extends every path into a predecessor by one edge. At the sinks this is $\max_u(r_u + \ell(u)) = L^*$.

(c) With $\tau_v = I_v$ we get $S_v = I_v + r_v$, so every stage adds $r_v + w_v$ along every path, and each edge adds δ.

(d) For every v, $r_v + \ell(v) \leq \ell'(v) \leq \ell'(e)$, because the path achieving $\ell(v)$ also has weight at least $r_v + \ell(v)$ under $r + w$, and e is an ancestor of every stage. □

Corollary 4.5 (Chains)**.** *For a chain of* d *stages with equal* r *and* w*,*

$$L_{\text{od}} = d(r + w) + (d - 1)\delta, \qquad L^* = r + d\,w + (d - 1)\delta.$$

The saving is $(d-1)\,r$*: the cascade of restores collapses to one restore. The speed-up* $\sigma(d) = L_{\text{od}}/L^*$ *increases with* d *and stays below* $1 + r/(w + \delta)$*.*

Proof. The formulas are Theorem 4.4(b, c) on a chain, where $r + \ell(v)$ is largest at the entry. Write $a = r + w + \delta$ and $b = w + \delta$; then $\sigma(d) = (ad - \delta)/(bd + r - \delta)$, and the numerator of its derivative in d is $a(r - \delta) + b\delta = r(r + w) > 0$. As $d \to \infty$, $\sigma(d) \to a/b = 1 + r/(w + \delta)$. □

With the running example's values, $\sigma(3) = 2.48$, $\sigma(8) = 4.60$, and the bound is 9.44.

Theorem 4.6 (Just-in-time look-ahead is latency-optimal and memory-minimal)**.** *(a) Every policy has* $M \geq M_{\min}$*, where* $M_{\min} := \sum_v m_v(r_v + w_v)$*.*

(b) The on-demand policy attains $M_{\min}$, but in general not L^*.

(c) The JIT policy is feasible ($\tau_v \geq 0$) and attains both L^* and $M_{\min}$.

Proof. (a) $F_v - \tau_v = (S_v - \tau_v) + w_v \geq r_v + w_v$, since $S_v \geq \tau_v + r_v$. (b) With $\tau_v = I_v$, $S_v = \tau_v + r_v$ exactly. (c) $S_v^{\text{eager}} = \max(r_v, I_v^{\text{eager}}) \geq r_v$, so $\tau_v \geq 0$. By induction in topological order, JIT reproduces the eager schedule: if all predecessors of v finish at their eager times, then $I_v = I_v^{\text{eager}}$, the sandbox is ready at $S_v^{\text{eager}} \geq I_v^{\text{eager}}$, and so $S_v = S_v^{\text{eager}}$. Hence $L = L^*$ by Theorem 4.4(b), and $S_v - \tau_v = r_v$ gives $M = M_{\min}$. □

So in this model there is no trade-off between latency and memory·time: just-in-time look-ahead Pareto-dominates both restoring on demand and restoring eagerly. It is not free: it holds its memory at different times, which raises the peak (Section 4.6).

When A2 fails. Restores that run at the same time may slow each other down (one disk, CPU for the restore, the container runtime). The next two results bound the loss.

Corollary 4.7 (Restore contention)**.** *Suppose that n restores started together all finish within $r\,(1 + \beta(n-1))$ for some $\beta \in [0,1]$. On a chain of d equal stages, eager look-ahead saves at least $(1-\beta)(d-1)\,r$ over on demand; the saving is never negative.*

Proof. On demand, the restore of stage v starts after stage $v-1$ has finished, so no two restores overlap and none is slowed. Eagerly, all d restores are ready by $R = r(1 + \beta(d-1))$, and by induction $F_v \leq R + v\,w + (v-1)\delta$. So $L_{\text{od}} - L \geq d\,r - R = (1-\beta)(d-1)\,r \geq 0$. □

$\beta = 0$ means fully parallel restores, $\beta = 1$ fully serial ones; the machine measurement of β is the first experiment of this thesis.

Corollary 4.8 (Random restore times)**.** *Let the restore times $r_1, \ldots, r_d$ of a chain be random. Then, pointwise, $L_{\text{eager}} \leq \max_j r_j + \sum_j w_j + (d-1)\delta$, while $L_{\text{od}} = \sum_j (r_j + w_j) + (d-1)\delta$. Hence $\mathbb{E}[L_{\text{od}} - L_{\text{eager}}] \geq d\,\mathbb{E}[r] - \mathbb{E}[\max_j r_j]$ for identically distributed r_j.*

Proof. All restores are ready by $\max_j r_j$; the induction of Theorem 4.7 with $R = \max_j r_j$ gives the bound. The on-demand latency is Theorem 4.4(c). □

Look-ahead pays the maximum of the parallel restores, on demand pays their sum. With lognormal restore times ($\sigma = 0.15$) and five stages, $\mathbb{E}[\max_j r_j] = 1.20\,\mathbb{E}[r]$.

4.3 Uncertain times and branches

Run times vary: in ORION's measurements the 95th percentile of a workflow's latency is up to 80× its 25th percentile [14]. Then a stage's input time I is a random variable and no trigger is right every time. Being ready too late delays the workflow; being ready too early holds memory idle.

Theorem 4.9 (The trigger under uncertainty is a newsvendor quantile)**.** *Let a stage's input time I have distribution function F. Let delay cost a per ms and idle memory cost $b = \mu\, m_v$ per ms. For a ready time $x = \tau + r_v$ the expected cost*

$$J(x) = a\,\mathbb{E}\big[(x - I)^+\big] + b\,\mathbb{E}\big[(I - x)^+\big]$$

is convex and minimised at $x^* = F^{-1}(\kappa)$ with $\kappa = b/(a+b)$. The optimal trigger is $\tau^* = \max(0, x^* - r_v)$.

Proof. $J'(x) = aF(x) - b\,(1 - F(x))$, which is zero where $F(x) = b/(a+b)$, and $J''(x) = (a+b)f(x) \geq 0$. Since J is convex, clipping at the feasible boundary $\tau \geq 0$ keeps the optimum. □

When memory is cheap ($\kappa \to 0$) the stage is ready at the earliest possible input: that is eager look-ahead. With equal prices it is ready at the median. ORION's convolution model [14] gives F for every stage, so Theorem 4.9 turns it into a trigger per stage without search.

Theorem 4.10 (Speculation on branches)**.** *Let stage s be reached with probability p, the branch be decided at time D, and τ_s^J be the JIT trigger of s if the branch is taken. Suppose a delay at s delays the workflow by the factor $\pi \in [0, 1]$.*

(a) If $D \le \tau_s^J$, restoring after the decision is as good as knowing the branch in advance: no speculation is needed.

(b) Otherwise, restore s speculatively at τ_s^J if and only if $p \ge b/(\pi a + b)$, which is κ when s is on the critical path.

Proof. (a) If the branch is taken, the restore can still start at τ_s^J. (b) Let $\Delta = D - \tau_s^J > 0$. Not speculating delays s by Δ when the branch is taken, costing $\pi a \Delta$ with probability p. Speculating holds a sandbox for Δ that is not needed when the branch is not taken, costing $b\Delta$ with probability $1 - p$. Compare $p\,\pi a\Delta$ with $(1 - p)\,b\Delta$. □

One price ratio thus decides both how early to restore and whether to restore speculatively.

4.4 Keep-alive

Platforms keep an idle sandbox alive for a time-to-live T after each call. Look-ahead is needed only when a workflow arrives cold.

Lemma 4.11 (The entry gate)**.** *Assume a uniform time-to-live T, no eviction under memory pressure, and invocations of the workflow that do not overlap. If the entry stage has a live sandbox when the workflow arrives, so does every stage.*

Proof. The previous invocation ran every stage, and every stage finished no earlier than the entry, since every stage descends from it: $F_v \ge F_e$. The arrival time t satisfies $t < F_e + T \le F_v + T$, and since invocations do not overlap, each sandbox is idle. □

So the platform does nothing extra when the entry is warm. The gate differs from checking every stage only under concurrency, eviction, or branches not taken last time.

Theorem 4.12 (Keep-alive cannot replace look-ahead for rare workflows)**.** *Let a workflow arrive as a Poisson process of rate λ and be kept alive for T after each call. Let L_{cold}, L_{warm} and L^* be its cold, warm and look-ahead latency, $\varepsilon = (L^* - L_{\text{warm}})/(L_{\text{cold}} - L_{\text{warm}})$, and let every stage have $r_v + w_v = c$. To match look-ahead's expected latency, keep-alive must hold at least $(1 - \varepsilon)\sum_v m_v$ on average, while just-in-time look-ahead without keep-alive holds $\lambda c \sum_v m_v$. The ratio $(1 - \varepsilon)/(\lambda c)$ is unbounded as $\lambda \to 0$.*

Proof. An arrival is cold when the gap before it exceeds T, with probability $e^{-\lambda T}$. Keep-alive's expected latency is $L_{\text{warm}} + e^{-\lambda T}(L_{\text{cold}} - L_{\text{warm}})$, which is at most L^* if and only if $e^{-\lambda T} \le \varepsilon$. In each gap $X \sim \text{Exp}(\lambda)$ the sandboxes are alive for $\min(X, T)$, with mean $(1 - e^{-\lambda T})/\lambda$; by the renewal reward theorem they are alive a fraction $1 - e^{-\lambda T} \ge 1 - \varepsilon$ of the time. Look-ahead holds $M_{\min} = c\sum_v m_v$ per invocation (Theorem 4.6), λ times per unit of time. □

With the thesis's measured three-stage Spring Boot workflow (12.1 s cold, 35 ms warm), $L^* = 0.93$ s and $c = 0.725$ s, so $\varepsilon = 0.074$. Look-ahead needs less memory below 1.28 calls per second; at one call per minute keep-alive needs 77× its memory, and at one per hour 4,600×. Matching look-ahead's 99th percentile would need $e^{-\lambda T} \le 0.01$: sandboxes alive 99% of the time.

4.5 Which stages get a snapshot, and how deep

A snapshot can be taken at different points: right after start-up, or after K warm-up requests (deeper snapshots leave less residual warm-up but may restore more slowly and take more storage), or not at all. Under restore on demand each stage costs its restore plus its residual warm-up, summed along paths; this is the objective of the depth optimisation in Chapter 3. Under look-ahead that objective is wrong, because restores no longer add up along paths (Theorem 4.4).

Setting. A *series-parallel* (SP) workflow is built from single stages by *series* composition $H_1; H_2$ (every exit of H_1 feeds every entry of H_2, with delay δ) and *parallel* composition $H_1 \parallel H_2$ (same input; the output waits for both). Workflow languages express exactly this (Step Functions' `Parallel`, Durable Functions' fan-out). Each stage v chooses an *option* $K \in \mathcal{K}_v$ with a provisioning time $p_v(K)$, a warm work $w_v(K)$ and a storage $s_v(K)$:

- a snapshot of depth K: $p = r_v(K)$, $w = R_v(K) + C_v$, $s = s_v(K)$;
- no snapshot: $p = A_v$ (cold start), $w = B_v + C_v$ (full warm-up plus steady run), $s = 0$.

Look-ahead starts every provisioning at time 0 (by Theorem 4.6, JIT timing gives the same finish times with the least memory). All-cold options give DAG-aware cold prewarming, as in Xanadu and ORION [3, 14], so the framework covers both.

For a sub-workflow H with its options fixed, let W_H be its longest warm path and

$$P_H = \max_{v \in H} (p_v + \ell_H(v)),$$

where $\ell_H(v)$ is the longest warm path from v to the exits of H.

Theorem 4.13 (Snapshot depth and restore timing, jointly)**.** *(a) If the input of H arrives at time x, its output is ready at $F_H(x) = \max(x + W_H,\ P_H)$, and the pair (W, P) composes as*

$$\begin{aligned} \textit{stage } v: &\quad W = w_v(K), && P = p_v(K) + w_v(K), \\ H_1; H_2: &\quad W = W_1 + \delta + W_2, && P = \max(P_1 + \delta + W_2,\ P_2), \\ H_1 \parallel H_2: &\quad W = \max(W_1, W_2), && P = \max(P_1, P_2). \end{aligned}$$

The workflow's latency is $\max(W, P) = P$ at the root, which equals L^*.

(b) Minimising the latency subject to $\sum_v s_v(K_v) \le S$ is solved exactly by dynamic programming over the SP decomposition tree, keeping for every sub-workflow the Pareto-minimal triples $(\textit{storage}, W, P)$. One number per sub-workflow does not suffice.

(c) If every chosen option has the same provisioning time r, the latency is $r + W$ at the root, and the problem is the on-demand depth optimisation with node weights $w_v = R_v(K) + C_v$ instead of $r_v + R_v(K) + C_v$.

(d) The problem is NP-hard even on chains, and the dynamic program is pseudo-polynomial in S.

Proof. (a) By induction on the SP tree. A stage whose input arrives at x starts at $\max(x, p)$ and is done at $\max(x + w, p + w)$. In series,

$$\begin{aligned} F(x) = F_2(F_1(x) + \delta) &= \max\big(\max(x + W_1, P_1) + \delta + W_2,\ P_2\big) \\ &= \max\big(x + W_1 + \delta + W_2,\ \max(P_1 + \delta + W_2, P_2)\big). \end{aligned}$$

In parallel, $F(x) = \max(F_1(x), F_2(x)) = \max(x + \max(W_1, W_2), \max(P_1, P_2))$. At the root $x = 0$; since $p \ge 0$, $P \ge W$, and unrolling P gives $\max_v(p_v + \ell(v)) = L^*$.

(b) The composition maps are non-decreasing in each of W_1, P_1, W_2, P_2, and storage adds up. So if one partial solution of a sub-workflow is no worse than another in storage, W and P, no completion of the worse one beats the same completion of the better one, and dominated triples can be discarded. The dynamic program enumerates all non-dominated combinations and is therefore exact. For the second claim, take a stage with two options of equal storage, c with $(p, w) = (990, 10)$, so $(W, P) = (10, 1000)$, and d with $(p, w) = (100, 500)$, so $(W, P) = (500, 600)$. Alone, d is faster ($600 < 1000$). Placed after a sub-workflow with $P_1 = 900$ (and $W_1 \leq 400$, $\delta = 0$), option c gives $\max(900 + 10, 1000) = 1000$ and d gives $\max(900 + 500, 600) = 1400$: a successor's warm work is added to its predecessor's finish, and a stage's own latency ignores that.

(c) If $p_v = r$ for every v, then $P = \max_v(r + \ell(v)) = r + W$.

(d) A chain under (c) is $\min \sum_v w_v(K_v)$ subject to $\sum_v s_v(K_v) \leq S$: the multiple-choice knapsack problem, which contains the knapsack problem. The dynamic program's state space is bounded by S for integer storages. □

Corollary 4.14 (Cold starts the workflow hides)**.** *Let stage v have a snapshot with $r_v < A_v$, and let the cold option have the same warm work. Switching v to a cold start leaves the latency unchanged if and only if $A_v + \ell(v) \leq L_{\text{rest}} := \max_{u \neq v}(p_u + \ell(u))$.*

Proof. ℓ does not depend on v's option, and $L = \max(p_v + \ell(v), L_{\text{rest}})$ by Theorem 4.13(a). □

Stages that qualify are late in the workflow (short remaining path $\ell(v)$) and have little warm-up to capture (CPython, Go). They need no snapshot and no storage: look-ahead cold-starts them behind upstream work.

Corollary 4.15 (Longest tail first)**.** *If the options change only the provisioning time, every snapshot takes the same storage and the budget allows k snapshots, then snapshotting the k stages with the largest $\ell(v)$ is optimal.*

Proof. ℓ does not depend on the choice and $L = \max_v(p_v + \ell(v))$. If $\ell(u) \geq \ell(v)$ and only v has a snapshot ($r \leq A$), swapping it to u gives $\max(r + \ell(u), A + \ell(v)) \leq A + \ell(u) \leq \max(A + \ell(u), r + \ell(v))$: never worse. Repeated swaps reach the k longest tails. □

Under on demand, every snapshot on the critical path saves the same $A - r$. Under look-ahead, the *position* in the workflow decides the value: the entry is worth most.

Corollary 4.16 (Prices and latency targets)**.** *Give every option a cost $c_v(K)$ per month: its storage price plus the resources a cold invocation spends in the stage, $m_v(p_v + w_v)$, times the number of cold invocations n_c. With a price a per ms of latency, minimise $J = n_c\, a\, L + \sum_v c_v$.*

(a) The optimum lies on the (cost, latency) Pareto front, which the dynamic program of Theorem 4.13(b) computes with cost in place of storage.

(b) It lies on the front's lower convex hull. The cheapest option set meeting a latency target is read off the same front.

(c) With Poisson arrivals of rate λ and time-to-live T, cold invocations occur at rate $\lambda e^{-\lambda T}$, which is largest at $\lambda = 1/T$, where it equals $1/(eT)$.

Proof. (a) J is non-decreasing in both coordinates. (b) A linear function on a finite set is minimised at a vertex of its convex hull. (c) A gap exceeds T with probability $e^{-\lambda T}$, and $\frac{d}{d\lambda}\lambda e^{-\lambda T} = (1 - \lambda T)e^{-\lambda T}$. □

Very frequent workflows stay warm and very rare ones are rarely called, so both gain little from a snapshot. With public list prices, a Java snapshot pays for itself in money above about 6–13 cold invocations a day, which only 6 of the 68 workflows of the Azure 2021 trace reach. Most snapshots buy latency, not money; the cost view decides where an image is stored and which snapshots to drop when their latency value is zero.

4.6 Memory

Theorem 4.6 bounds memory·time, not the peak. A platform has a hard memory budget, and look-ahead restores later stages while earlier ones run.

Theorem 4.17 (Peak memory, memory slots and the guard)**.** *(a) On a chain of d equal stages the on-demand peak is m and the JIT peak is $m \cdot \min\bigl(d, \lceil (r+w)/(w+\delta) \rceil\bigr)$.*

(b) Let at most k sandboxes be alive at once, admitted in stage order. Then on a chain $S_v = r + (v-1)(w+\delta)$ for $v \le k$ and

$$S_v = \max\bigl(S_{v-k} + r + w,\ S_{v-1} + w + \delta\bigr) \quad (v > k).$$

The latency is non-increasing in k; $k = 1$ gives $d(r+w)$, never worse than on demand; the latency is L^* if and only if $k \ge \min\bigl(d, \lceil (r+w)/(w+\delta) \rceil\bigr)$; and on long chains each stage costs $\max\bigl(w+\delta, (r+w)/k\bigr)$.

(c) (The guard.) *On any workflow with a budget $C \ge \max_v m_v$, let restores queue for memory: stages whose input has arrived first, then by trigger time, with strict head-of-line admission; a stage whose input has arrived and does not fit preempts admitted stages whose input has not arrived (latest trigger first), which re-queue. Then the budget is never exceeded; there is no deadlock; if C is at least the JIT schedule's peak, the latency is L^* with no preemption; and on a chain, look-ahead under the guard is never slower than on demand under the same budget.*

Proof. (a) Under JIT stage v holds memory on $[(v-1)(w+\delta), (v-1)(w+\delta) + r + w)$: intervals of length $r+w$ starting every $w+\delta$, of which at most $\lceil (r+w)/(w+\delta) \rceil$ contain any instant. On demand the intervals are disjoint.

(b) Stages finish in order, so the slot for v is freed by stage $v-k$, and $S_v = \max(S_v^*, F_{v-k} + r, F_{v-1} + \delta)$ with $S_v^* \le S_{v-1} + w + \delta$, which is the recurrence. Monotonicity in k follows by induction from $S_{v-k-1} \le S_{v-k}$. Substituting $S_{v-1} = S_{v-k} + (k-1)(w+\delta)$ shows that no stall occurs if and only if $k(w+\delta) \ge r + w$. Once a stall occurs it persists, because S^* advances by exactly $w+\delta$ per stage and S by at least that. During a stall $S_{v+k} = S_v + r + w$, so the schedule is periodic with k stages per $r+w$.

(c) Running stages always finish, and every other holder of memory is a look-ahead sandbox whose input has not arrived, which can be preempted: so a stage whose input has arrived is eventually admitted, and there is no deadlock. Admission never exceeds C. If C covers the JIT peak, every admission happens at its trigger time, exactly as in the JIT schedule. On a chain, when the input of v arrives, $v-1$ has finished and every other holder is a look-ahead sandbox, so v is admitted at once and $S_v \le I_v + r_v$; induction on I gives the claim. □

For the eight-stage Java chain, $k = 1, 2, 3, 4, 8$ sandboxes give 5.80, 2.98, 2.25, 1.68 and 1.26 s, against 5.81 s on demand (Figure 4.2). The schedule of (b) is also optimal among all schedules with k slots: the exact solver of Section 4.7 finds nothing faster on any instance tested (check A3). This optimality is verified by computation, not proved.

On general workflows, look-ahead under the guard is faster than on demand under the same budget on average (1.94× over random workflows), but not always: on 12% of random workflows it is slower at some budget, by up to 1.37×. This is the classical anomaly of scheduling under a resource limit: making tasks faster can lengthen the schedule [10]. Section 4.7 removes it by planning the triggers exactly.

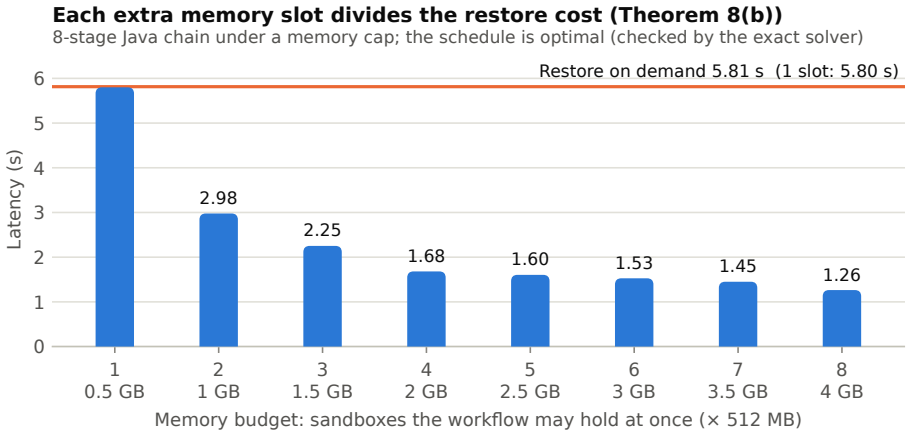


Figure 4.2: Latency of an eight-stage Java chain under look-ahead when at most k sandboxes may be alive at once (Theorem 4.17(b); $r = 650\,\text{ms}$, $w = 75\,\text{ms}$, $\delta = 2\,\text{ms}$). With one slot it equals restoring on demand; each extra slot divides the restore cost.

4.7 The problem behind the approach

This section shows that scheduling snapshot restores in a workflow is a classical operations-research problem, identifies the results above as its tractable special cases, and gives the exact method used for the rest.

4.7.1 A sandbox is an activity, an edge is a time lag

Lemma 4.18 (No idle sandbox)**.** *Some optimal schedule, under any memory budget, never lets a restored sandbox wait for its input.*

Proof. If $\tau_v + r_v < I_v$, move the trigger to $\tau'_v = I_v - r_v$. The stage still starts at I_v, so no finish time changes, and it holds memory on $[\tau'_v, F_v) \subset [\tau_v, F_v)$, so the memory in use never increases. □

So stage v becomes one *activity* of fixed length $p_v = r_v + w_v$ that holds m_v for its whole duration, and each edge (u, v) becomes a start-to-start *time lag*:

$$\tau_v \geq \tau_u + p_u + \delta - r_v \qquad \text{(look-ahead)} \tag{4.1}$$

$$\tau_v \geq \tau_u + p_u + \delta \qquad \text{(on demand)} \tag{4.2}$$

$$\tau_v \geq 0 \qquad \text{(the workflow arrives at 0).}$$

Look-ahead is exactly the shortening of every precedence lag by the successor's restore time. The lag can be negative: v's restore may start before u starts. Two consequences follow. Every schedule without idle sandboxes holds the same memory·time $\sum_v m_v(r_v + w_v) = M_{\min}$; schedules differ only in *when* they hold it. And the earliest-start schedule of the lag network (4.1) is the JIT schedule of Theorem 4.6, while that of (4.2) is the on-demand schedule. In the running example the look-ahead lag is $727 - 650 = 77\,\text{ms}$ instead of $727\,\text{ms}$.

4.7.2 Which known problem

Table 4.2 maps the model to project scheduling. The problem is the *multi-mode resource-constrained project scheduling problem with generalised precedence relations*, MRCPSP/max [5]; with the modes fixed it is RCPSP/max [1, 16]. Table 4.3 lists its special cases and where each result of this chapter sits.

Table 4.2: The model as a project-scheduling problem.

project scheduling	snapshot restores in a workflow
project	one cold workflow invocation
activity	one stage's sandbox: restore and run, length $r_v + w_v$
mode of an activity	cold start; snapshot of depth K, stored locally or remotely
generalised precedence (time lag)	edge: $\tau_v \geq \tau_u + p_u + \delta - r_v$
renewable resource	memory (budget C); restore channels (Theorem 4.21)
non-renewable resource	snapshot storage, or money
makespan	workflow latency

Table 4.3: Special cases of the problem, their complexity, and the results of this chapter.

case	known problem	complexity	this chapter
no budget, modes fixed	temporal scheduling: longest paths in the lag network	$O(\|V\| + \|E\|)$	Theorems 4.4 and 4.6
+ random durations	stochastic PERT	per stage	Theorem 4.9
+ branches	probabilistic networks	per branch	Theorem 4.10
choose modes, no budget	discrete time–cost trade-off	strongly NP-hard in general [4]; SP reduction [6]	Theorem 4.13: $(\text{storage}, W, P)$ dynamic program
budget, equal chain	RCPSP/max, special structure	closed form	Theorem 4.17(b)
budget, general	RCPSP/max with one resource	strongly NP-hard (Theorem 4.19)	exact branch and bound (Algorithm 1); the guard as a heuristic
keep-alive between calls	online weighted caching	online	Theorem 4.22; GDSF [2, 8]
CPU for start-ups, no budget	flexible resource profiles [15]; preemptive deadline scheduling [11]	polynomial: max-flow and bisection	Theorem 4.26; one spare core: Theorems 4.29 and 4.32

Proposition 4.19 (Hardness)**.** *Minimising the latency of one workflow under a memory budget is NP-hard in the strong sense.*

Proof. With $r_v = 0$ (a stage that is already warm), $\delta = 0$ and $m_v = 1$ for all v, and $C = k$, the lags (4.1) become finish-to-start precedences and the budget allows k activities at once: this is $P \mid \text{prec} \mid C_{\max}$, which is strongly NP-hard already with unit processing times [17]. □

4.7.3 The exact solver

Algorithm 1 is the standard branch and bound on minimal forbidden sets [1]. A *forbidden set* is a set of activities that run at the same instant and together exceed the budget; it is minimal if removing any activity makes it fit.

Proposition 4.20 (Exactness)**.** *Algorithm 1 returns the minimal latency under the budget C.*

Proof. Intervals on a line that overlap pairwise share a common point (the Helly property). So in any schedule that respects the budget, some two activities of F do not overlap: one ends before the other starts, and the loop covers every such choice. The earliest-start schedule of each branch is the pointwise earliest schedule meeting its constraints, so its makespan bounds every schedule in the branch from below, and pruning at *best* is safe. □

In pure Python on one core, Algorithm 1 solves random workflows of up to 8 stages in about 1 ms at the median (0.1 s at worst for chains, 1.3 s for general DAGs) and 10-stage DAGs in

Algorithm 1 EXACT: minimal latency of one workflow under a memory budget C.

best $\leftarrow$ latency of the memory guard (or of capped on demand, if lower)
SEARCH($\emptyset$)
procedure SEARCH($\mathcal{E}$) ▷ $\mathcal{E}$: added constraints "u ends before v starts"
    $\tau \leftarrow$ earliest-start schedule of the lag network plus $\mathcal{E}$ ▷ longest paths
    if τ does not exist (positive cycle) **or** makespan(τ) $\geq$ *best* **then**
        return
    $F \leftarrow$ a minimal forbidden set of τ
    if there is none **then**
        best $\leftarrow$ makespan(τ); **return** ▷ feasible: a better schedule
    for all ordered pairs (u, v) of distinct activities in F **do**
        SEARCH($\mathcal{E} \cup \{\tau_v \geq \tau_u + p_u\}$)

21 ms at the median, with a heavy tail (12 s at the 95th percentile). Real workflows are small: in Azure's workflows the median depth is 3 and the 95th percentile 8 [14]. The platform runs the solver with a node budget and keeps the best schedule found, which is never worse than the guard's, because the search starts from it.

Against this optimum, the guard is optimal on 67–77% of instances with 3–8 stages and 3–5% slower on average, but the gap grows with size (a mean of 14% for 10-stage DAGs) and reaches 87% in the worst case (Figure 4.3b). The guard reacts to whichever stage is ready; the plan knows which stage is the long pole. In the smallest example we found, a Python entry feeds a Java–Java chain and a machine-learning stage under a 1 GB budget: the guard gives the memory to the ML stage, whose input arrives first, and takes 1.93 s; the optimum makes the ML stage wait, although its input is there, because it can finish behind the chain, and takes 1.29 s.

4.7.4 Restore contention as a second resource

Restores that slow each other (Theorem 4.7) fit the same model as a second renewable resource: c restore channels, each restore holding one channel for r_v.

Corollary 4.21 (Restore channels)**.** *On a chain of equal stages with c restore channels and no memory budget, the schedule $\tau_v = \max(\tau_{v-1} + w + \delta,\ \tau_{v-c} + r)$ is feasible, and on long chains each stage costs $\max(w + \delta, r/c)$. It equals the exact solver's optimum on every instance tested (check A3).*

Each channel divides the restore cost, as each memory slot does in Theorem 4.17(b). For a five-stage Java chain (on demand 3.63 s), $c = 1, 2, 4, 9$ give 3.33, 2.02, 1.38 and 1.03 s. A measured contention β behaves roughly like $c \approx 1/\beta$ channels.

4.7.5 The algorithm

The platform runs four procedures. Algorithm 2 runs at deployment and reads no user data; Algorithms 3 and 4 run for each cold workflow arrival; keep-alive runs between calls.

Between calls, idle sandboxes are evicted per function by GreedyDual-Size-Frequency [2, 8, 13]: priority = clock + frequency × restore cost / memory. The simulator showed that each rule of Algorithms 3 and 4 is needed: without the likely-path rule, the queue, the re-planning, or the two rules that switch planning off, the planner loses to the guard in some setting.

Keep-alive under look-ahead. Keep-alive policies value a warm sandbox at its own cold-start cost. For a cold arrival served by look-ahead that valuation is wrong:

Proposition 4.22 (Keep-alive is a threshold decision per workflow)**.** *Let $\mathcal{W}$ be the set of stages with an idle warm sandbox when a workflow with one entry arrives. Then*

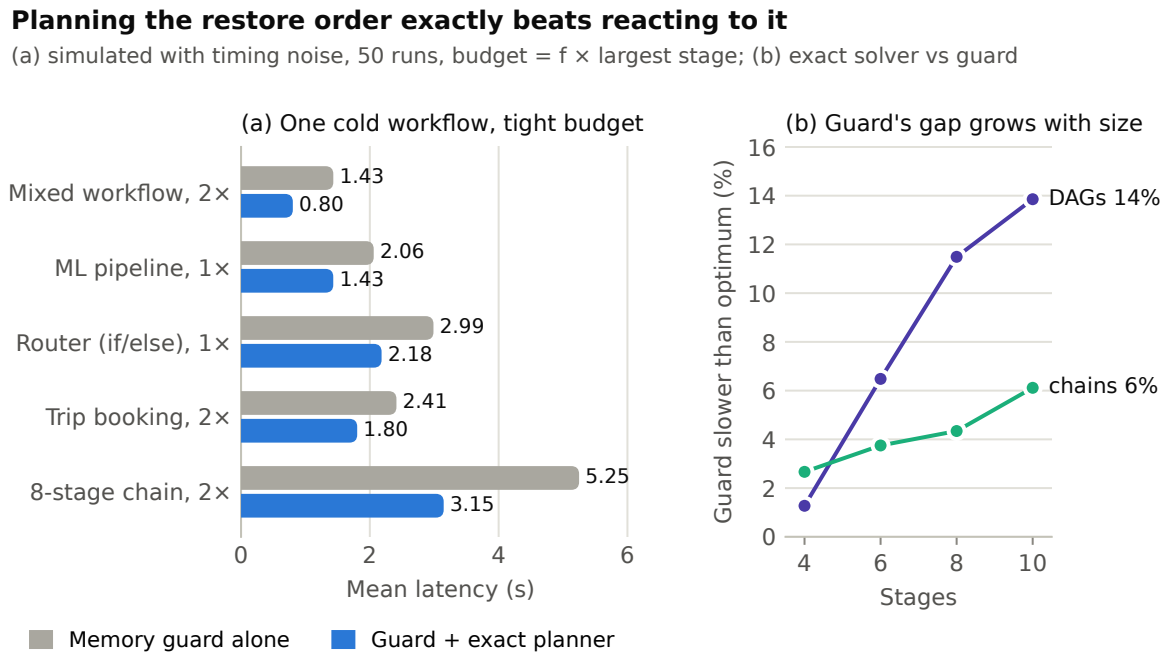


Figure 4.3: (a) One cold workflow under a tight memory budget, simulated with timing noise: the memory guard alone against the guard executing an exact plan. (b) The guard's distance from the optimum on random workflows, by size.

Algorithm 2 BUILD (per function, at each deployment).

1: profile with the developer's test requests at the serving CPU size: A, B, $r(K)$, $s(K)$, m
2: choose snapshot points: after start-up; for JIT runtimes also after warm-up, stopping when the JIT's compile counters flatten (Theorem 4.23)
3: modes per stage: cold | snapshot(K) local | snapshot(K) remote
4: choose modes with the (storage, W, P) dynamic program (Theorem 4.13); pick a point of the front by storage budget, price or latency target (Theorem 4.16)
5: checkpoint after reset hooks have re-seeded random state and removed secrets

(a) its look-ahead latency is $L(\mathcal{W}) = \max\big(\ell(e),\ \max_{v \notin \mathcal{W}}(r_v + \ell(v))\big)$;

(b) the best k *stages to keep warm are the* k *with the largest* $r_v + \ell(v)$;

(c) on a chain of d *equal stages, keeping the first* $j < d$ *warm saves* $\min(r, j(w+\delta))$, *and keeping all* d *warm saves* r.

Proof. (a) is Theorem 4.4 with $r_v = 0$ for the warm stages. (b) By (a) the latency depends only on the largest $r_v + \ell(v)$ left outside $\mathcal{W}$, so a top-k set is best. (c) $\ell(1) - \ell(j+1) = j(w+\delta)$. □

On a five-stage Java chain, keeping only the entry warm saves 77 ms, not the 650 ms a per-function policy assumes. Tested as an eviction policy on the Azure trace, however, evicting whole workflows lost to per-function GDSF: most arrivals are hot, the entry gate then skips look-ahead, and a missing stage costs its full restore, which is the per-function valuation. The proposition therefore stays a statement about cold arrivals, and the platform uses GDSF.

4.8 When to stop warming up

Build-time warm-up needs a stopping rule that does not depend on what the requests contain.

Algorithm 3 PLAN (per cold workflow arrival).

set $r_v \leftarrow 0$ for every stage that has an idle warm sandbox
keep the likely path: stages reached with probability at least $1/2$
$\tau \leftarrow$ earliest start of the lag network (4.1) ▷ the JIT schedule, Theorem 4.6
$C_w \leftarrow$ memory this workflow may use now (free, plus idle sandboxes it may evict)
if peak$(\tau) \le C_w$ **then return** τ ▷ optimal
if the platform is over budget, or was within the last minute **then return** τ
$\tau' \leftarrow$ EXACT(G, C_w) with a node budget ▷ Algorithm 1
if latency$(\tau') \ge$ the on-demand latency **then return** τ
return τ'

Algorithm 4 RUN (event-driven, shared by all workflows on a machine).

on trigger τ_v**:**
    restore v; if it does not fit, v waits in a queue until memory frees
on input of v**:**
    start v on its restored sandbox; a planned stage whose trigger is later waits for it
on deadline $\tau_v + r_v$ passed while v waits**:**
    fall back to the guard of Theorem 4.17(c): a demand start that may preempt unclaimed look-ahead sandboxes
on any stage finishes**:**
    if the rest of a planned workflow now fits in the memory it can get, drop its plan and use JIT triggers

Proposition 4.23 (Depth is work, not requests)**.** *Under threshold tiered compilation without deoptimisation, method μ reaches tier t after a set $\mathcal{P}$ of warm-up requests if and only if $c_\mu(\mathcal{P}) \ge \theta_t$, where $c_\mu(\mathcal{P}) = \sum_{x \in \mathcal{P}} n_\mu(x)$ counts its invocations. Hence $\mathcal{P} \subseteq \mathcal{P}'$ implies that every method compiled after $\mathcal{P}$ is compiled after $\mathcal{P}'$: adding requests never un-compiles. The number of requests is not a sufficient statistic; the counters are.*

Proof. The counters are additive over requests and thresholds are fixed, so c_μ is monotone in $\mathcal{P}$. Two sets of equal size can differ in c_μ by the ratio of per-request work. □

Receiver-type pollution and deoptimisation can make a larger warm-up set compile slower code, so the performance consequence is empirical. The rule: stop warming up when the JIT's compile counter stops rising (in HotSpot, from `hsperfdata` or JFR). In our measurements the compiled-method count at the snapshot rose monotonically with warm-up work.

4.9 Planning CPU for start-ups

So far every start-up had a fixed length: a restore took r_v. That length depends on the CPU the sandbox gets. Starting a function is CPU work (restoring memory, loading classes, compiling hot methods), and serverless functions get small CPU shares. In our measurements the same JVM warm-up took 3.59 s at 0.25 vCPU and 0.86 s at 1 vCPU, for the same CPU time (0.90 and 0.86 CPU-seconds; Chapter 3). A server's spare CPU can therefore shorten start-ups without costing CPU time. But it is limited, and a cold workflow starts several sandboxes at once, so who gets it, and when, decides how fast the workflow starts. Cloud Run's startup CPU boost gives every starting instance the same extra CPU [9]; ORION sizes each stage once, for its whole run [14]. This section makes the CPU of each start-up a decision over time, planned from the workflow graph. The results of Section 4.2 are its special case with no spare CPU.

4.9.1 Model

Definition 4.24 (CPU plan)**.** Each stage v of a workflow (Theorem 4.1) has a *start-up* of CPU work $U_v > 0$, in CPU-seconds: a snapshot restore plus the first request's residual warm-up, or a cold boot plus the full warm-up. The start-up can use at most $c_v > 0$ cores (its *cap*), and it has a *private quota* $q_v \in [0, c_v]$ that no other start-up can use. It may begin at its *release* $a_v \geq 0$; under look-ahead $a_v = 0$. The server has *spare CPU* $\Pi(t) \geq 0$, piecewise constant: the CPU the running requests leave. A *CPU plan* gives each start-up a rate $x_v(t) = y_v(t) + z_v(t)$ for $t \geq a_v$, with a private part $0 \leq y_v(t) \leq q_v$, a shared part $z_v(t) \geq 0$, $x_v(t) \leq c_v$, and $\sum_v z_v(t) \leq \Pi(t)$. Start-up v is *ready* at the time ρ_v at which $\int_{a_v}^{\rho_v} x_v = U_v$. The schedule is that of Theorem 4.2 with $\tau_v + r_v$ replaced by ρ_v: $S_v = \max(\rho_v, I_v)$ and $F_v = S_v + w_v$.

The platform can enforce a plan: it sets each starting sandbox's CPU limit (cgroup `cpu.max`) and lowers it to the function's normal share once the start-up is done. The plan used by the simulator sets $q_v = 0$: a start-up is guaranteed nothing, so a start-up that is not urgent cannot hold CPU that an urgent one needs. Table 4.4 lists the assumptions.

Table 4.4: Assumptions of the CPU plan.

assumption	statement	status
B1 linear speed-up	a start-up given rate $x \leq c_v$ does its work U_v in U_v/x	measured: JVM warm-up 4.2× faster from 0.25 to 1 vCPU at the same CPU time; on the test machine (Chapter 3) a cold JVM start 4.2× and a snapshot restore 3.5–4.5× faster, at the same CPU-seconds within 6–12%, and the speed-up continues to about $c_v = 2$ cores
B2 separate requests	running requests keep their CPU share; start-ups use only $\Pi(t)$	the platform's choice (a CPU limit per sandbox)
B3 known work	U_v and w_v are profiled	as A2 and A3; under uncertainty use quantiles (Theorem 4.9)
B4 look-ahead	a start-up may begin when its workflow arrives	as in Section 4.2

4.9.2 Deadlines per start-up

Lemma 4.25 (Deadlines per start-up)**.** *For every CPU plan the latency is* $L = \max_{v \in V}(\rho_v + \ell(v))$. *Hence* $L \leq \Lambda$ *if and only if every start-up is ready by its* deadline $D_v(\Lambda) := \Lambda - \ell(v)$.

Proof. As in the proof of Theorem 4.4(b), with r_v replaced by ρ_v. By induction in topological order, $F_v = \max_{u,P}(\rho_u + W(P))$ over the ancestors u of v (including v) and the paths P from u to v: at the entry $F_e = \rho_e + w_e$ because $I_e = 0 \leq \rho_e$, and for $v \neq e$ the term $\rho_v + w_v$ covers $u = v$ while $\max_{u \in \mathrm{pred}(v)} F_u + \delta + w_v$ extends every path into a predecessor by one edge. At the sinks this is $\max_u(\rho_u + \ell(u))$. The equivalence holds because L is a maximum of the terms $\rho_v + \ell(v)$. □

The lemma turns "finish the workflow soon" into "every start-up meets its own deadline". The deadlines depend on the target Λ only through a common shift, and the most urgent start-ups are those with the longest remaining path $\ell(v)$.

4.9.3 The exact plan

Theorem 4.26 (The exact CPU plan)**.** *Fix a target* Λ *and let* $D_v = \Lambda - \ell(v)$. *Cut the time axis at every release, every deadline and every breakpoint of* Π *into slices* $I_1, I_2, \ldots$, *and let* Π_k *be the*

spare CPU on I_k. Build a network with a source, a sink, one node per start-up and one node per slice, and the arcs

- *source $\to v$, capacity U_v;*
- *$v \to$ sink, capacity $q_v \max(0, D_v - a_v)$ (the private quota);*
- *$v \to k$ for every slice $I_k \subseteq [a_v, D_v]$, capacity $(c_v - q_v)\,|I_k|$;*
- *$k \to$ sink, capacity $\Pi_k\,|I_k|$.*

Then a CPU plan with latency at most Λ exists if and only if a maximum flow saturates every source arc, and such a flow gives the plan. Feasibility is monotone in Λ, so bisection on Λ finds the optimal latency L^*_Π to any precision.

Proof. By Theorem 4.25, latency at most Λ means $\rho_v \le D_v$ for every v.

- *A flow gives a plan.* Let g_v be the flow on v's private arc and f_{vk} on its arc to slice k. Run v at private rate $g_v/(D_v - a_v) \le q_v$ over its window and at shared rate $f_{vk}/|I_k| \le c_v - q_v$ in slice k. The shared rates in a slice sum to at most Π_k, the total rate is at most c_v, and the work done by D_v is $g_v + \sum_k f_{vk} = U_v$.
- *A plan gives a flow.* Integrate the private part y_v over the window and the shared part z_v over each slice. Every capacity holds, and the flow out of v is the work done by D_v, which is U_v.
- *Monotonicity.* A larger Λ moves every deadline later, so a plan for Λ is a plan for every $\Lambda' > \Lambda$. □

This is Horn's flow construction for preemptive scheduling with release times and deadlines [11], with a rate cap per job. In project-scheduling terms, the start-ups are activities whose duration depends on how much of a resource they get: project scheduling with flexible resource profiles [15]. The problems of Section 4.7 fix the durations.

Corollary 4.27 (A certificate)**.** *No CPU plan reaches Λ if and only if some set T of start-ups has*

$$\sum_{v \in T} U_v \;>\; \sum_{v \in T} q_v \max(0, D_v - a_v) + \int_0^\infty \min\Big(\Pi(t),\; \sum_{v \in T,\; a_v \le t < D_v} (c_v - q_v)\Big)\, dt.$$

Proof. Max-flow/min-cut. A cut that keeps the start-ups of T on the source side cuts the source arcs of all others, the private arcs of T, and in each slice either the arcs from T or the slice's arc to the sink, whichever is smaller. Its capacity is therefore $\sum_{v \notin T} U_v$ plus the right-hand side, and the flow saturates every source arc if and only if no such cut has capacity below $\sum_v U_v$. □

Such a set T proves that the target cannot be met; it is a lower bound that needs no search.

Proposition 4.28 (The two extremes)**.** *Let every $a_v = 0$.*

(a) With no spare CPU ($\Pi \equiv 0$) and $q_v > 0$, the optimal latency is $\max_v(U_v/q_v + \ell(v))$: the L^* of Theorem 4.4 with $r_v = U_v/q_v$.

(b) With spare CPU at least $\sum_v (c_v - q_v)$ at all times, it is $L_{\text{ideal}} = \max_v(U_v/c_v + \ell(v))$.

(c) The optimal latency is non-increasing in Π.

Proof. (a) Without shared CPU, start-up v is ready no earlier than U_v/q_v, and running every start-up at its quota from time 0 attains that; Theorem 4.25 gives the maximum. (b) Every start-up can run at its cap from time 0, and none can be ready before U_v/c_v. (c) A plan for Π is a plan for any $\Pi' \ge \Pi$. □

So the CPU plan interpolates between the fixed-CPU look-ahead of Theorem 4.4 ($\Pi = 0$) and the ideal, in which every start-up runs at full speed.

4.9.4 One spare core

Proposition 4.29 (One spare core)**.** *Let $q_v = 0$, $a_v = 0$ and $\Pi(t) \le c_v$ for all v and t: the spare CPU never exceeds what one start-up can use.*

(a) Running the start-ups one at a time, earliest deadline first (longest $\ell(v)$ first), each at rate $\Pi(t)$, is optimal.

(b) On a chain of d stages with constant Π, the optimum L^*_Π and restoring on demand with the same boost, $L_{\mathrm{od},\Pi} = \sum_v (U_v/\Pi + w_v) + (d-1)\delta$, satisfy

$$0 \le L_{\mathrm{od},\Pi} - L^*_\Pi \le \sum_{v<d} w_v + (d-1)\,\delta,$$

with equality when the chain is CPU-bound, that is when $L^*_\Pi = \sum_v U_v/\Pi + w_d$.

Proof. (a) Number the start-ups so that $\ell(v_1) \ge \ell(v_2) \ge \dots$ and let $\bar\Pi(t) = \int_0^t \Pi$. Earliest deadline first makes v_k ready at ρ_k, the first time $\bar\Pi$ reaches $U_{v_1} + \dots + U_{v_k}$. In any plan the rates sum to at most $\Pi(t)$, so the first k start-ups cannot all be ready before ρ_k: some v_i with $i \le k$ is ready at $\rho'_{v_i} \ge \rho_k$, and $\ell(v_i) \ge \ell(v_k)$. By Theorem 4.25 that plan's latency is at least $\rho_k + \ell(v_k)$ for every k, which is the latency of earliest deadline first. This is Jackson's rule [12].

(b) Restoring on demand is a plan, so $L^*_\Pi \le L_{\mathrm{od},\Pi}$. In any plan the last start-up to be ready, say v, is ready no earlier than $\sum_u U_u/\Pi$, and $\ell(v) \ge w_d$ on a chain, so $L^*_\Pi \ge \sum_u U_u/\Pi + w_d$ by Theorem 4.25. Subtracting gives the bound, with equality exactly when this lower bound is attained. □

With about one core to spare, look-ahead can only overlap start-ups with the stages' own run times, $\sum(w+\delta)$: about 0.1 s on a three-stage Java chain at 0.25 vCPU. This is why, in the simulator, the CPU plan only ties with the startup boost on one spare core. The parallelism of look-ahead needs more than one spare core.

4.9.5 Branches

Proposition 4.30 (Leftover speculation)**.** *If the start-ups of stages that may not run (behind an if/else not yet decided) get only the CPU that the certain start-ups' plan leaves, every certain start-up is ready exactly when it would be without them.*

Proof. The certain start-ups' rates are computed without the speculative ones, which use only what is left at each instant. □

Proposition 4.31 (The cost of a wrong guess)**.** *Under the conditions of Theorem 4.29 with constant Π, let a speculative start-up s be served as if certain, ahead of some certain start-ups in earliest-deadline order, and let its branch not be taken. Then every certain start-up after it is ready U_s/Π later, and a CPU-bound chain finishes exactly U_s/Π later.*

Proof. One start-up runs at a time at rate Π, so inserting U_s of work before them shifts each later ready time by U_s/Π. When the chain is CPU-bound its latency is the last ready time plus w_d. □

So serving a branch reached with probability p as certain pays in expectation only if $p\,G \ge (1-p)\,U_s/\Pi$, where G is what its early start saves when the branch is taken. By Theorem 4.29(b), G is at most the overlap with the run times, $\sum(w+\delta)$ along its path. With the measured Java values on one spare core ($U_s/\Pi \approx 0.7$ s, $G \approx 0.05$ s) this needs $p \ge 0.93$. The rule has the form of Theorem 4.10 (speculate if and only if $p \ge \kappa$), with CPU time as the cost. Hence the online rule below: with at most one spare core only stages certain to run are served as certain, and with more the likely path is, as in Algorithm 3.

Algorithm 5 BOOST (event-driven, per machine): CPU for start-ups.

on a start-up or a request begins or ends, or a branch is decided**:**
    requests run at their quota; $\Pi \leftarrow$ the machine's CPU that they leave
    give each start-up the deadline $t_w + \Lambda_w - \ell(v)$ ▷ Theorem 4.25
    certain $\leftarrow p = 1$ if $\Pi \le$ one cap, else $p \ge 1/2$ ▷ Theorem 4.31
    if the certain start-ups belong to one workflow **then**
        constant rates that minimise their largest lateness; share the rest equally
    else
        earliest deadline first, each up to its cap ▷ Theorem 4.32
    the other start-ups share what is left ▷ Theorem 4.30
    set each starting sandbox's CPU limit to its rate

4.9.6 Bursts

Proposition 4.32 (Bursts on one spare core)**.** *Workflows w arrive at times t_w, with $q_v = 0$ and $\Pi(t) \le c_v$. Give each start-up v of workflow w the release t_w and the deadline $t_w + \Lambda_w - \ell(v)$, where Λ_w is the ideal latency of w (Theorem 4.28(b)). Then preemptive earliest deadline first minimises the largest delay of any workflow over its ideal,* $\max_w(L_w - \Lambda_w)$.

Proof. By Theorem 4.25 applied to each workflow, $L_w - \Lambda_w = \max_{v \in w}(\rho_v - D_v)$, so $\max_w(L_w - \Lambda_w)$ is the largest lateness over all start-ups. Each start-up can absorb all the spare CPU, so in the time scale $\bar{\Pi}(t) = \int_0^t \Pi$ this is preemptive scheduling on one machine with release times, minimising the largest lateness, for which preemptive earliest deadline first is optimal [11]. □

With more than one spare core, greedy earliest deadline first is not optimal (the multiprocessor case [7]). The exact plan then comes from Theorem 4.26, with one common shift added to all deadlines and minimised by bisection.

4.9.7 The online rule

The exact plan needs the whole future. The platform runs a rule that is recomputed at every event (Algorithm 5).

For one workflow the rule chooses, by bisection on the lateness, the constant rates that minimise the largest lateness of its start-ups; they are optimal among plans whose rates stay constant from now on. The exact plan can do better by giving CPU to the urgent start-ups first; on our test workflows in a fluid model the rule is within 0–9% of it, while an equal split, the startup boost's rule, is up to 1.39× slower. For several workflows the rule is earliest deadline first: optimal for the largest delay on one spare core (Theorem 4.32), and chosen by simulation beyond that. Not proved, and not claimed: a competitive ratio for the online rule, the burst rule beyond one spare core, and anything about real speed-up curves (B1).

4.10 Verification

Each result of this chapter was checked by an independent computation in a verification program (`theory/verify_dag.py`, standard-library Python). Each check compares the result with a computation that does not use it: exhaustive search, brute force over all options, an event-level simulation, or Monte Carlo sampling. Each check is paired with a *control*: a variant designed to fail, which passes only if the check detects the violation. A check that cannot fail proves nothing. The program runs 93 checks, 18 of them controls, and all pass. Tables 4.5 and 4.6 list the main ones.

Table 4.7 states the status of each kind of claim. What cannot be proved, and is not claimed, are the absolute restore times and the contention β: they are properties of the hardware, measured in Chapter 6.

4.11 Summary

Restoring every stage of a cold workflow just in time, $\tau_v = S_v^* - r_v$, reaches the lowest latency any policy can reach and holds the least memory·time; on a chain it turns d restores into one. Under uncertainty the trigger is a quantile, and branches are restored ahead when their probability exceeds the same price ratio. Which stages deserve a snapshot, and how deep, is decided exactly by a dynamic program with two numbers per sub-workflow; under look-ahead, a stage's position in the workflow decides its value. Look-ahead raises the memory peak, and under a budget each extra sandbox divides the restore cost.

The problem as a whole is resource-constrained project scheduling with time lags. The problem class, the branch and bound and the list-scheduling heuristic are textbook operations research. What this thesis contributes is the modelling (Theorem 4.18 turns a workflow of snapshot restores into that problem), the special cases solved exactly and proved (Theorems 4.4, 4.6, 4.9, 4.10, 4.13 and 4.17), the (W, P) state of the dynamic program, the keep-alive analysis, and the measurements that calibrate the model. Timing provisioning along the workflow graph is Xanadu's idea [3]; this thesis applies it to snapshots, whose restore is short enough to hide, and proves what is optimal.

A start-up is CPU work, and its length depends on the CPU it gets. Planning the server's spare CPU per start-up gives each start-up a deadline from the workflow graph (Theorem 4.25), and the best plan is a maximum flow (Theorem 4.26). The fixed-CPU look-ahead of Theorem 4.4 is its case with no spare CPU. With about one spare core, earliest deadline first is optimal, and look-ahead gains only the stages' own run times over starting on demand with the same CPU (Theorem 4.29); the planning pays where several start-ups compete for more than one spare core.

Table 4.5: Main checks in `theory/verify_dag.py`. "Control" is the variant that must fail.

result	check	what is compared	control
Theorem 4.4	T1	eager = L^*, on demand = $\ell'(e)$ on 2000 random DAGs; 150,000 random policies never below L^*	triggers before the arrival ($\tau < 0$) do beat L^*
Theorem 4.5	C1.1	closed forms, monotone speed-up and its bound, 384 parameter sets	
Theorem 4.7	C1.2	the saving bound, analytically and in the simulator's contention model	
Theorem 4.8	C1.3	18,006 Monte-Carlo instances	comparing with the deterministic bound fails
Theorem 4.6	T2	2000 DAGs; 30,000 random policies never below $M_{\min}$; the simulator's JIT reproduces L^* and $M_{\min}$	eager holds strictly more
Theorem 4.9	T3	the κ-quantile vs the numerical optimum, 3 distributions $\times$ 4 price ratios	the quantile $a/(a+b)$ costs more
Theorem 4.10	T4	exact expected costs, 228 cases	
Theorem 4.11	L5	gated = ungated in the simulator, sequential calls	overlapping calls and an if/else workflow differ
Theorem 4.12	T6	renewal Monte Carlo	
Theorem 4.13	T7	composition rules vs the expanded DAG (2000 SP workflows); the dynamic program vs brute force (600)	a one-number state is wrong on 63/600
Theorems 4.14 and 4.15	T7	600 instances each	shortest tails first is worse
Theorem 4.16	C7.3	front vs brute force; lower hull; $\lambda e^{-\lambda T}$	counting every call as cold ranks wrongly
Theorem 4.17	T8	peak (2000 chains); recurrence vs slot simulation (1500); the guard (1500)	without preemption the guard deadlocks (582/1500)
Theorem 4.18, Theorem 4.20	A1, A2	lag network = JIT and on demand (1000 DAGs); branch and bound vs brute force over all integer triggers (200)	
Theorem 4.17(b) optimal, Theorem 4.21	A3	exact solver vs the closed forms (200 chains each)	
the guard's gap (Section 4.7)	A4	the exact optimum is never above the guard; gap statistics over 3000 capped instances	the guard is not optimal everywhere (838/3000)
Algorithms 3 and 4	A5, A6	the guard executing the plan reaches the optimum with no preemption (800); the simulator's planner = the exact solver	
Theorem 4.22	P9	brute force over all k-sets	the per-function valuation overstates the saving for 99% of stages

Table 4.6: Checks of the CPU plan (Section 4.9) in `theory/verify_dag.py`.

result	check	what is compared	control
Theorem 4.25	CP1	the latency formula and the deadlines, 2000 DAGs with random ready times	
Theorems 4.26 and 4.27	CP2	max-flow vs the exhaustive subset condition (3000 instances, with and without private quotas); the plan meets every deadline and a smaller target is infeasible (500 DAGs)	ignoring the caps changes the answer (172/500); an equal split is worse (398/500)
Theorem 4.28	CP3	no and unlimited spare CPU, 500 DAGs each	
Theorem 4.29	CP4	earliest deadline first = the exact plan (500 DAGs); the chain bound (250 chains)	one start-up at a time is worse with more than one spare core (443/500)
Theorems 4.30 and 4.31	CP5, CP6	500 CPU-bound chains each	
Theorem 4.32	CP7	earliest deadline first = the exact smallest largest lateness (250 bursts)	greedy earliest deadline first is not optimal beyond one spare core (171/500)

Table 4.7: What is proved, what is verified, and what must be measured.

claim	status
look-ahead is latency-optimal; JIT is also memory-minimal; trigger and speculation rules	proved; verified on random DAGs and against the simulator
joint depth and timing on SP workflows; exact dynamic program; hidden cold starts	proved; verified against brute force
peak memory, k slots, the guard's safety	proved
optimality of the k-slot and channel schedules	verified by the exact solver
the capped problem is RCPSP/max; exactness of the branch and bound	proved; verified against brute force
the guard's distance from the optimum; the planner under noise and load	measured on random DAGs; simulated
A2: restores run in parallel (β small)	assumption, to be measured
A3: residual warm-up after a restore	assumption, to be measured
deadlines per start-up; the exact CPU plan; the fixed-CPU and ideal extremes; one spare core; speculation; bursts on one spare core	proved; verified against exhaustive enumeration and the exact plan
the online CPU rule's distance from the exact plan; the burst rule beyond one spare core	measured in a fluid model; chosen by simulation
B1: start-ups speed up linearly with CPU up to one core	measured for JVM warm-up; assumption for restores and cold starts, to be measured
no user data in any snapshot	by construction

Bibliography

[1] M. Bartusch, R. H. Möhring, and F. J. Radermacher. Scheduling project networks with resource constraints and time windows. *Annals of Operations Research*, 16:199–240, 1988.

[2] L. Cherkasova. Improving WWW proxies performance with Greedy-Dual-Size-Frequency caching policy. Technical Report HPL-98-69, HP Laboratories, 1998.

[3] N. Daw, U. Bellur, and P. Kulkarni. Xanadu: Mitigating cascading cold starts in serverless function chain deployments. In *Middleware*, 2020.

[4] P. De, E. J. Dunne, J. B. Ghosh, and C. E. Wells. Complexity of the discrete time-cost tradeoff problem for project networks. *Operations Research*, 45(2):302–306, 1997.

[5] B. De Reyck and W. Herroelen. The multi-mode resource-constrained project scheduling problem with generalized precedence relations. *European Journal of Operational Research*, 119(2):538–556, 1999.

[6] E. L. Demeulemeester, W. S. Herroelen, and S. E. Elmaghraby. Optimal procedures for the discrete time/cost trade-off problem in project networks. *European Journal of Operational Research*, 88(1):50–68, 1996.

[7] S. K. Dhall and C. L. Liu. On a real-time scheduling problem. *Operations Research*, 26(1):127–140, 1978.

[8] A. Fuerst and P. Sharma. FaasCache: Keeping serverless computing alive with greedy-dual caching. In *ASPLOS*, 2021.

[9] Google Cloud. Startup CPU boost. Cloud Run documentation. Accessed September 2026.

[10] R. L. Graham. Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2):416–429, 1969.

[11] W. A. Horn. Some simple scheduling algorithms. *Naval Research Logistics Quarterly*, 21(1):177–185, 1974.

[12] J. R. Jackson. Scheduling a production line to minimize maximum tardiness. Research Report 43, University of California, Los Angeles, 1955.

[13] Q. Liu, Y. Cheng, H. Shen, et al. Concurrency-informed orchestration for serverless functions. In *ASPLOS*, 2025.

[14] A. Mahgoub, E. B. Yi, K. Shankar, S. Elnikety, S. Chaterji, and S. Bagchi. ORION and the three rights: Sizing, bundling, and prewarming for serverless DAGs. In *OSDI*, 2022.

[15] A. Naber and R. Kolisch. MIP models for resource-constrained project scheduling with flexible resource profiles. *European Journal of Operational Research*, 239(2):335–348, 2014.

[16] K. Neumann, C. Schwindt, and J. Zimmermann. *Project Scheduling with Time Windows and Scarce Resources*. Springer, 2nd edition, 2003.

[17] J. D. Ullman. NP-complete scheduling problems. *Journal of Computer and System Sciences*, 10(3):384–393, 1975.